Coding for Artificial Intelligence

Problem Set 8

Packaging a Module and Automated Style Linting

Name: Tanishka Hira
Roll No.: A26MTAI0001

16 September 2026

Objective

The objective of this practical was to convert the `search4letters` function into a locally installable Python package and use `pycodestyle` to identify intentional PEP 8 style violations.

The package was then built as a source distribution, installed using `pip`, tested from outside the package directory, and finally checked using automated style linting.

1 Task 1: Creating the Distribution Package

The working package folder was named:

```
text_utility_pkg
```

The folder contained the Python module, setup file, README file, distribution folder, and a separate copy of the module used for intentional style-error testing.

Package Structure

```
text_utility_pkg/
|
|-- vsearch.py
|-- CopyOfvsearch.py
|-- setup.py
|-- README.txt
|
```

```
|-- dist/
    |-- vsearch-1.0.tar.gz
```

vsearch.py

The module contained the `search4letters()` function. The function accepts a phrase and a collection of target letters and returns the letters that occur in both.

```
def search4letters(phrase: str,
                   letters: str = 'aeiou') -> set:
    """Return the set of letters found in phrase."""
    return set(letters).intersection(set(phrase))
```

The function uses Python sets and the `intersection()` operation.

For example:

```
search4letters('TGGACC', 'GC')
```

returns the common letters:

```
{'C', 'G'}
```

Installing the Build Tool

Before generating the distribution package, the Python `build` package was installed/upgraded using:

```
python -m pip install --upgrade build
```

The installation completed successfully.

Updating pip

During the process, `pip` was also updated using:

```
python.exe -m pip install --upgrade pip
```

The update completed successfully.

A warning similar to the following appeared:

```
WARNING: Ignoring invalid distribution ~ip
```

However, this warning did not prevent the package from being built or installed.

Building the Source Distribution

The source distribution was created using:

```
python -m build --sdist
```

The build process created the distribution file:

```
dist/vsearch-1.0.tar.gz
```

The terminal confirmed the successful build with:

```
Successfully built vsearch-1.0.tar.gz
```

This file represents the packaged version of the `vsearch` module.

Installing the Generated Package

The generated source distribution was installed locally using:

```
python -m pip install dist/vsearch-1.0.tar.gz
```

The package was successfully built into a wheel and installed.

The important confirmation message was:

```
Successfully installed vsearch-1.0
```

Therefore, the `vsearch` module was now available as an installed Python package rather than merely as a local Python file.

Verifying the Installed Package

To ensure that Python was importing the installed package rather than simply reading the local `vsearch.py` file, the verification was performed from a directory outside `text_utility_pkg`.

Python was opened from:

```
C:\Users\HP\Desktop>
```

The following commands were then executed:

```
import vsearch

vsearch.search4letters('TGGACC', 'GC')
```

The obtained result was:

```
{'C', 'G'}
```

This confirmed that the installed `vsearch` package could be imported and that the `search4letters()` function was working correctly.

Testing the module from outside the original package directory was important because otherwise Python could directly find the local `vsearch.py` file. In that case, successful importing would not necessarily prove that the generated package had been installed correctly.

2 Task 2: Diagnostic Linting and PEP 8 Refactoring

The second part of the practical involved deliberately introducing coding-style violations and detecting them using `pycodestyle`.

A separate working copy named:

```
CopyOfvsearch.py
```

was used so that the original working module remained separate.

Intentional Flaw Injection

Three intentional PEP 8 errors were introduced.

The modified file was conceptually as follows:

```
def search4letters(phrase: str, letters:str = 'aeiou') -> set:
    """Return the set of letters found in phrase."""
    return set(letters).intersection(set(phrase))#SPACE #SPACE #SPACE

def say_hello():
    return 'Hello!!'
```

The three intentional errors were:

(i) A missing space after the colon in the type hint:

```
letters:str
```

instead of:

```
letters: str
```

(ii) Trailing whitespace was placed at the end of one line, after the final visible character.

(iii) Only one blank line was placed between the two top-level function definitions instead of the two blank lines required by PEP 8.

Running pycodestyle

The linter was executed from the package directory using:

```
python -m pycodestyle CopyOfvsearch.py
```

The following errors were detected:

```
CopyOfvsearch.py:1:42: E231 missing whitespace after ':'
CopyOfvsearch.py:3:50: W291 trailing whitespace
CopyOfvsearch.py:5:1: E302 expected 2 blank lines, found 1
```

Thus, all three deliberately introduced style violations were successfully identified by `pycodestyle`.

Meaning of the Detected Errors

- **E231 – Missing whitespace after ':'**

  This error occurred because the parameter was written as:

  ```
  letters:str
  ```

  PEP 8 requires a space following the colon in a type annotation:

```
letters: str
```

- **W291 – Trailing whitespace**

  This warning indicates that one or more unnecessary spaces occurred after the final visible character on a line.

  The spaces were removed so that the line ended immediately after the Python statement.

- **E302 – Expected 2 blank lines, found 1**

  PEP 8 requires two blank lines between top-level function definitions.

  Initially, only one blank line separated `search4letters()` and `say_hello()`.

  An additional blank line was therefore added.

Corrected Version

After fixing the detected issues, the file became:

```
def search4letters(phrase: str, letters: str = 'aeiou') -> set:
    """Return the set of letters found in phrase."""
    return set(letters).intersection(set(phrase))

def say_hello():
    return 'Hello!!'
```

The corrections included:

- adding the missing space after the colon,
- removing trailing whitespace, and
- inserting two blank lines between the top-level functions.

Final Style Verification

After correcting the file, the same command was executed again:

```
python -m pycodestyle CopyOfvsearch.py
```

This time, no messages were displayed and the command prompt returned immediately.

No output from `pycodestyle` indicates that no style violations were detected.

Verification Using the Exit Code

The command exit status was then checked using:

```
echo Exit code: %ERRORLEVEL%
```

The result was:

```
Exit code: 0
```

An exit code of `0` confirms that `pycodestyle` completed successfully and reported no remaining violations.

Therefore, the corrected file passed the automated style check.

3 Submission Note

The `vsearch` module was successfully converted into a locally installable Python package.

The `build` package was installed and a source distribution was generated using:

```
python -m build --sdist
```

This produced:

```
dist/vsearch-1.0.tar.gz
```

The generated package was then installed through `pip`. Its installation was verified by importing `vsearch` from outside the original `text_utility_pkg` directory and executing `search4letters()` successfully.

For the linting task, a separate file named `CopyOfvsearch.py` was intentionally modified to contain three PEP 8 violations.

Running:

```
python -m pycodestyle CopyOfvsearch.py
```

successfully detected the `E231`, `W291`, and `E302` violations.

After correcting these issues, another linting run produced no output. The command status was verified using `%ERRORLEVEL%`, which returned an exit code of `0`.

Hence, both the Python packaging process and the automated PEP 8 style-checking process were completed successfully.

Conclusion

This practical demonstrated the complete process of converting a Python module into a locally installable package, generating a source distribution, installing it with `pip`, and verifying that the installed module could be imported correctly.

It also demonstrated the use of `pycodestyle` as an automated linting tool. Three intentional PEP 8 violations were introduced, successfully detected, understood, and corrected.

The final linting run produced no errors and returned an exit code of `0`, confirming that the corrected Python file passed the style check.